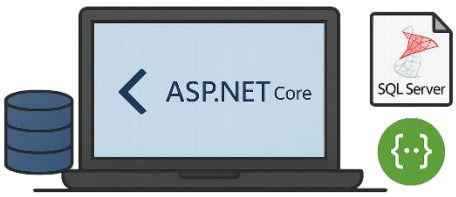
Building a Complete ASP.NET Core Web API with SQL Server and Swagger



1. Install the Required Software

1.1 Install SQL Server (Developer Edition)

1. Download from: https://www.microsoft.com/en-us/sql-server/sql-server-downloads
2. Choose **Developer Edition**.
3. During setup:
    - Choose **Basic installation** or **Custom**.
    - Instance name: keep default (MSSQLSERVER).
    - Authentication type: **Windows Authentication**.
4. After installation, note your **Server Name** (example: DESKTOP-XXXXXXX).

1.2 Install SQL Server Management Studio (SSMS)

1. Download from: https://aka.ms/ssmsfullsetup
2. Install normally.
   SSMS is the graphical interface used to manage and query SQL Server databases.

1.3 Install Visual Studio 2022 (Community Edition)

1. Download from: https://visualstudio.microsoft.com/downloads/
2. During installation, select the **ASP.NET and web development** workload.

---

2. Create a Database and Table in SQL Server

2.1 Open SSMS and Connect

1. Open **SQL Server Management Studio**.
2. In the connection window:

- Server name: your system name (for example DESKTOP-XXXXXXX).
- Authentication: **Windows Authentication** and **connect**.

2.2 Create the Database and Table

1. Click **New Query**.
2. Paste and execute the following SQL script:

```
CREATE DATABASE WEBAPI_DB;
GO

USE WEBAPI_DB;
GO

CREATE TABLE Employees
(
    ID INT PRIMARY KEY IDENTITY,
    FirstName NVARCHAR(50),
    LastName NVARCHAR(50),
    Gender NVARCHAR(50),
    Salary INT
);
GO

INSERT INTO Employees VALUES ('Pranaya', 'Rout', 'Male', 60000);
INSERT INTO Employees VALUES ('Anurag', 'Mohanty', 'Male', 45000);
INSERT INTO Employees VALUES ('Preety', 'Tiwari', 'Female', 45000);
INSERT INTO Employees VALUES ('Sambit', 'Mohanty', 'Male', 70000);
INSERT INTO Employees VALUES ('Shushanta', 'Jena', 'Male', 45000);
INSERT INTO Employees VALUES ('Priyanka', 'Dewangan', 'Female', 30000);
INSERT INTO Employees VALUES ('Sandeep', 'Kiran', 'Male', 45000);
INSERT INTO Employees VALUES ('Shudhansshu', 'Nayak', 'Male', 30000);
INSERT INTO Employees VALUES ('Hina', 'Sharma', 'Female', 35000);
INSERT INTO Employees VALUES ('Preetiranjan', 'Sahoo', 'Male', 80000);
GO
```


```
CREATE DATABASE WEBAPI_DB

GO

USE WEBAPI_DB

GO

CREATE TABLE Employees

(

  ID int PRIMARY KEY IDENTITY(1,1),

  FirstName nvarchar(50),

  LastName nvarchar(50),

  Gender nvarchar(50),

  Salary int

)

GO

INSERT INTO Employees VALUES

('Pranaya', 'Rout', 'Male', 60000),

('Anurag', 'Mohanty', 'Male', 45000),

('Preety', 'Tiwari', 'Female', 45000),

('Sambit', 'Mohanty', 'Male', 70000),

('Shushanta', 'Jena', 'Male', 45000),

('Priyanka', 'Dewangan', 'Female', 30000),

('Sandeep', 'Kiran', 'Male', 45000),

('Shudhansshu', 'Nayak', 'Male', 30000),

('Hina', 'Sharma', 'Female', 35000),

('Preetiranjan', 'Sahoo', 'Male', 80000)

GO
```

3. Create an ASP.NET Core Web API Project

3.1 Create the Project

1. Open **Visual Studio**.
2. Select **Create a new project**.
3. Choose **ASP.NET Core Web API**.
4. Configure the project:
    - Project name: EmployeeService
    - Framework: .NET 8.0 (Long Term Support)
    - Authentication type: None
    - Check **Use Controllers**
    - Check **Enable OpenAPI support (Swagger)**
    - Check **Configure for HTTPS**
    - Uncheck container and Docker options
5. Click **Create**.

---

4. Install Entity Framework Core (ORM)

Entity Framework Core (EF Core) is the Object-Relational Mapper (ORM) used in .NET. It performs the same role as JPA/Hibernate in Java.

4.1 Install EF Core Packages

1. In Visual Studio, open:
   Tools → NuGet Package Manager → Package Manager Console
2. Run the following commands

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer

Install-Package Microsoft.EntityFrameworkCore.Tools
```

These install EF Core and its SQL Server provider.

5. Create the Model Class

1. In **Solution Explorer**, create a new folder named Models.
2. Inside the folder, create a new C# class file named **Employee.cs**.

```
namespace EmployeeService.Models
{

    public class Employee
    {

        public int ID { get; set; }

        public string FirstName { get; set; }
        2 references
        public string LastName { get; set; }
        2 references
        public string Gender { get; set; }
        2 references
        public int Salary { get; set; }
    }
}
```

---

6. Create the Database Context

1. Create a new folder named Data.
2. Inside it, create a new class file named **AppDbContext.cs**.

```
using Microsoft.EntityFrameworkCore;

namespace EmployeeService.Models
{
    5 references
    public class EmployeeDbContext : DbContext
    {
        0 references
        public EmployeeDbContext(DbContextOptions<EmployeeDbContext> options) : base(options)
        {
        }

        6 references
        public DbSet<Employee> Employees { get; set; }
    }
}
```

---

7. Configure the Connection String

1. Open the file **appsettings.json**.
2. Replace the content with the following:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=DESKTOP-88KVFBE;Database=WEBAPI_DB;Trusted_Connection=True;TrustServerCertificate=True;"
  },

  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },

  "AllowedHosts": "*"
}
```

Replace DESKTOP-XXXXXXX with your actual SQL Server name.

---

8. Configure EF Core in Program.cs

1. Open **Program.cs**.
2. Replace its content with the following code:

```
using EmployeeService.Models;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Connect to SQL Server
builder.Services.AddDbContext<EmployeeDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

// Middleware
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

---

9. Create the EmployeesController:

1. In the Controllers folder, create a new C# class file named **EmployeesController.cs**.

- GET /api/employees – Retrieve all employees

```
// ☑ GET: api/employees
[HttpGet]
0 references
public async Task<IActionResult> GetEmployees()
{
    var employees = await _context.Employees.ToListAsync();
    return Ok(employees);
}
```

- GET /api/employees/{id} – Retrieve one employee by ID

```
// ☑ GET: api/employees/{id}
[HttpGet("{id}")]
1 reference
public async Task<IActionResult> GetEmployee(int id)
{
    var emp = await _context.Employees.FindAsync(id);
    if (emp == null)
        return NotFound(new { Message = "Employee not found" });

    return Ok(emp);
}
```

- POST /api/employees – Create a new employee

```
// ☑ POST: api/employees
[HttpPost]
0 references
public async Task<IActionResult> CreateEmployee([FromBody] Employee emp)
{
    if (emp == null)
        return BadRequest();

    _context.Employees.Add(emp);
    await _context.SaveChangesAsync();

    return CreatedAtAction(nameof(GetEmployee), new { id = emp.ID }, emp);
}
```

- PUT /api/employees/{id} – Update an existing employee

```
// ☑ PUT: api/employees/{id}
[HttpPut("{id}")]
0 references
public async Task<IActionResult> UpdateEmployee(int id, [FromBody] Employee emp)
{
    if (id != emp.ID)
        return BadRequest(new { Message = "ID mismatch" });

    var existingEmp = await _context.Employees.FindAsync(id);
    if (existingEmp == null)
        return NotFound(new { Message = "Employee not found" });

    existingEmp.FirstName = emp.FirstName;
    existingEmp.LastName = emp.LastName;
    existingEmp.Gender = emp.Gender;
    existingEmp.Salary = emp.Salary;

    _context.Entry(existingEmp).State = EntityState.Modified;
    await _context.SaveChangesAsync();

    return Ok(existingEmp);
}
```

- DELETE /api/employees/{id} – Delete an employee

```
// ☑ DELETE: api/employees/{id}
[HttpDelete("{id}")]
0 references
public async Task<IActionResult> DeleteEmployee(int id)
{
    var emp = await _context.Employees.FindAsync(id);
    if (emp == null)
        return NotFound(new { Message = "Employee not found" });

    _context.Employees.Remove(emp);
    await _context.SaveChangesAsync();

    return Ok(new { Message = $"Employee with ID {id} deleted successfully" });
}
```

---

10. Run and Test the API

1. Press **Ctrl + F5** to start the project.
2. The browser will open the **Swagger** interface automatically.
3. You will see the available endpoints.

11. Verification in SSMS

1. Open SQL Server Management Studio.
2. Run the following query:
3. You should see the inserted and updated records from the Web API

```
USE WEBAPI_DB;

SELECT * FROM Employees;
```